

Stage 3 Report

Flow Matching Before a Frozen Linear Classifier

Yousef Shihade · Mira Bitar

2026

A flow-matching layer is placed between the frozen image feature and the linear classifier trained in Stage 1, which is then held fixed. The flow is initialised so that it is exactly the identity, meaning the untrained system is bit-for-bit the Stage 1 linear probe — so every result reported here is caused by training the flow and nothing else.

Two training strategies are compared against that probe and against each other, over 225 training runs.

The three headline results

- **The flow helps, modestly** — **+0.1 to +2.6 pp** over the trained probe. Stage 2 predicted this: its headroom rule says the flow recovers what training the classifier would have gained, and here the classifier is already trained.
- **The two strategies are indistinguishable on accuracy** (+0.13 pp, $p = 0.80$) but **not on robustness** — classifier-guided training survives a 10× larger learning rate ($p = 0.048$).
- **Unfreezing the classifier changes nothing** — −0.02 pp, $p = 0.97$ — which is what licenses the frozen-classifier framing of everything else.

Stage 3 at a glance

What was run, what it produced, and where every number lives.

The system

$z \rightarrow [\text{FM}, T = 4 \text{ Euler steps}] \rightarrow \hat{z} \rightarrow [\text{frozen } W, b] \rightarrow \text{logits}$

- The **encoder** is frozen (Stage 1). The **classifier** is frozen (Stage 1, trained). The **flow** is the only trainable component.
- The flow is initialised to the **exact** identity, so at epoch 0 the system *is* the Stage 1 linear probe — $\Delta = 0.00$ pp, predictions identical.

What was run

Step	What it does	Runs	Time
01 Setup	Recover and freeze the Stage 1 classifier; verify identity init	—	—
02 Strategy 1	End-to-end rolled-out classification + regularisation sweep	81	32 min
03 Strategy 2	Classifier-guided targets + the brief's four-knob sweep	99	16 min
04 Evaluation	The required comparison — retrain nothing	—	—
05 Visualisation	Joint PCA; geometry measured in the full feature space	6	81 s
06 Joint (optional)	Unfreeze the classifier + a classifier-only control	45	13 min

Protocol, fixed in step 01 and unchanged throughout

- T = 4** Euler steps · **K = 10** · seeds **{0, 1, 2}** — the same k-shot subsets, and therefore the same images, as Stages 1 and 2.
- Raw features, no L2 normalization** — the opposite of Stage 2, and the one decision Stage 3 inherits differently. Measured, not assumed (page 4).
- Velocity network**: Stage 2's design unchanged — 2×512 hidden, SiLU, t concatenated — with the output layer zeroed.
- Validation-accuracy checkpointing**, which is Stage 1's own rule, so the baseline and both strategies are selected identically.

Reproducibility. 52 tests pass across Stages 2 and 3. Every figure and table in this report is generated by a tracked notebook and backed by a tracked CSV; each step folder carries an `\_artifact\_manifest.csv` listing exactly what it wrote. Stages 1 and 2 are **not modified** — verified against git.

1 · What Stage 3 asks

The setup, the constraints that make it a real experiment, and the two strategies.

The goal

- Stage 1 built a **linear probe**: freeze the encoder, extract features z , train $s = Wz + b$ on $K = 10$ images per class. That is as good as a *linear* rule on those features can be.
- Stage 3 asks a sharper question: **the features are frozen and possibly badly shaped — can we learn to reshape them so the classifier we already have does better?**

Four constraints from the brief

- **Train the classifier first, exactly as in Stage 1, then freeze it.** It never learns again, so it cannot adapt to whatever the flow does — the flow must move features toward what the classifier *already* wants.
- **Initialise the FM close to identity**, so that before training the whole system behaves like the original probe. This is what makes “ Δ vs. probe” mean anything.
- **One representative encoder per dataset, one training-set size, one T** throughout.
- **Compare three things per dataset**: the Stage 1 probe, end-to-end rolled-out training, and classifier-guided training.

The two strategies are genuinely different answers

	Strategy 1 — end-to-end	Strategy 2 — classifier-guided
Loss	$CE(W\hat{z} + b, y)$	standard FM: $\ v(z_t, t) - u\ ^2$
Gradient path	through all T Euler steps	never through the rollout
Classifier's role	it <i>is</i> the loss	it <i>builds a target</i> , then ordinary FM
Main risk	backprop through T composed steps	targets go stale between refreshes

Strategy 2's idea in one line: roll $z \rightarrow \hat{z}$, take gradient steps on $\hat{z}$ **in feature space** to get a version the classifier likes better, then train the flow to transport $z \rightarrow \hat{z}$ with a completely standard FM objective. The classifier says *where to go*; the FM machinery does the *going*.

Datasets and encoders — carried unchanged from Stage 1

Combination	Classes	K = 10 train	Probe accuracy	Role
ResNet-18 / DTD	47	470	52.04%	required
DINOv2 / FGVC-Aircraft	100	1000	51.54%	required
ResNet-18 / FGVC-Aircraft	100	1000	27.87%	extra

The third combination is beyond what the brief requires. It is included because Stage 1 measured its features as the most entangled of the nine (own-vs-other cosine margin 0.041), making it the most informative test of whether a flow can rescue a representation a linear classifier struggles with.

2 · The one decision Stage 3 had to make differently

Stage 2 L2-normalized its features. Stage 3 must not — and the cost is measured.

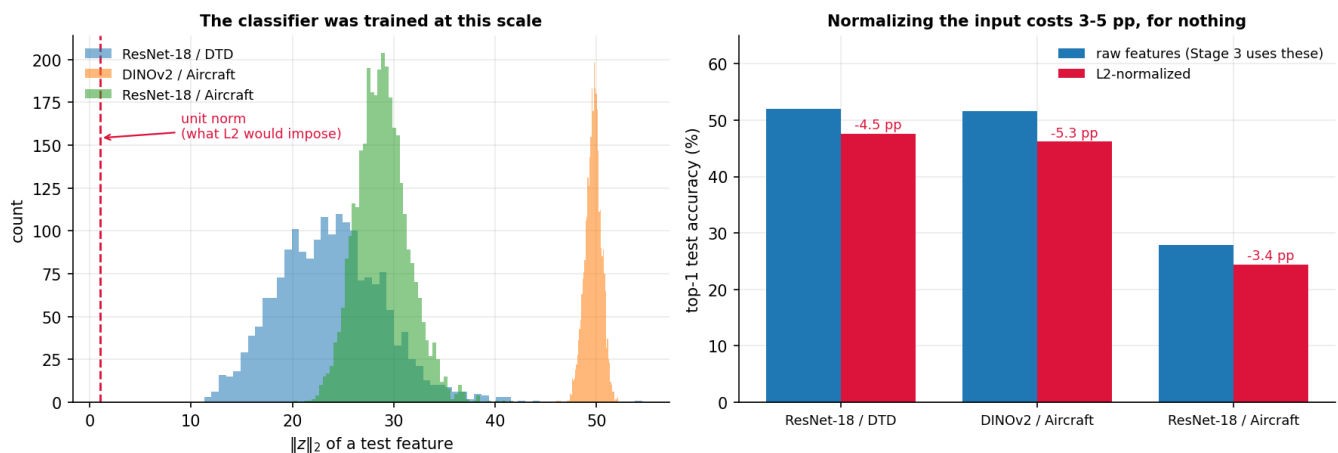
Why the stages differ

- **Stage 2 normalized** because its targets were class prototypes, which are unit-norm by construction, and its decision rule was **cosine similarity** — scale-invariant.
- **Stage 3 must not**, because its frozen classifier was **trained on raw features** at norms of 24–50, and an affine rule is **not** scale-invariant: $W(z/\|z\|) + b$ rescales Wz but leaves b untouched, so the decision boundary moves relative to the data.

What normalizing would cost

Encoder / dataset	mean $\ z\ $	raw	L2-normalized	cost
ResNet-18 / DTD	23.7	52.04%	47.55%	−4.49 pp
DINOv2 / FGVC-Aircraft	49.7	51.54%	46.19%	−5.34 pp
ResNet-18 / FGVC-Aircraft	28.9	27.87%	24.44%	−3.43 pp

- **Not a collapse** — normalizing preserves each feature's *direction*, where most class information lives — but a free loss of 3.4–5.3 pp on every combination.
- **The DINOv2 row is the sharpest form of the argument.** Its norms are already tightly clustered (46.9–52.0), so normalizing is close to dividing every feature by the same constant — and it still costs the most of the three. A near-uniform rescale is not harmless to an affine classifier.
- This is also why the identity initialisation is **load-bearing** rather than book-keeping: a flow initialised the ordinary way would displace features on the very first forward pass, and the classifier in front of it has no slack for that.



Feature-norm distributions per combination, and the measured accuracy cost of feeding the frozen classifier L2-normalized features instead of raw ones.

3 · Step 01 — Recovering and freezing the classifier

No flow is trained. Four things everything downstream stands on, each measured rather than assumed.

The blocker: Stage 1 never saved the classifier

- Stage 1 saved accuracies, training curves and per-image predictions — but **not W and b** . The trained layer went out of scope, because Stage 1 only reported numbers and Stage 2 used prototypes instead.
- **Stages 1 and 2 are finished and published, so neither was modified.** Stage 3 carries its own ``train_frozen_classifier``, which reproduces Stage 1's loop and additionally returns the weights.
- The hyperparameters are **not** duplicated — ``ProbeConfig`` is imported from Stage 1, so learning rate, weight decay, batch size, epoch budget and checkpoint rule keep exactly one definition in the project.
- Duplicated logic is normally a liability. Two things stop it being one: this step **asserts** the re-derived probes reproduce Stage 1 exactly, and an 11-test equivalence suite runs both implementations against each other on accuracy, predictions, selected epoch and **every** training curve.

What was verified

Check	Result
Re-derived probes vs. Stage 1's published accuracies (9 runs)	max $\Delta = 0.00$ pp
Test predictions, per image	identical — 0 of 16,920 differ
Identity init: max $ z - z $ over all test features	0.0 — bit-for-bit
Accuracy at initialisation vs. the probe	$\Delta = 0.00$ pp
Classifier parameters requiring grad	0
W.grad after a full backward pass through the system	None

“Frozen” is mechanical, not a promise

- ``FrozenClassifier`` sets ``requires_grad=False`` at construction **and** overrides ``train()`` to refuse to leave eval mode — so a parent module calling ``train()`` on the whole system cannot silently unfreeze it.
- Gradient must still flow **through** the layer, because Strategy 1 differentiates the loss back into the flow behind it. That is a different thing from the classifier's own parameters being updated, and both are pinned by tests.

A consequence of exact identity, worth stating

- Only the readout is zeroed; hidden layers keep their random initialisation. A hidden layer's gradient is proportional to the readout weight above it, so **while the readout is exactly zero the hidden layers receive exactly zero gradient** — on the first optimiser step only the readout moves.
- Measured: hidden $|grad| = 0.0$ at step 0, then $3.7e-03$ and $7.7e-03$ at steps 1 and 2, with the loss falling throughout. Standard behaviour for a zero-initialised readout, and the price of starting *exactly* at identity rather than near it.

4 · Step 02 — Strategy 1: end-to-end rolled-out training

81 runs = 9 configurations × 3 combinations × 3 seeds. ~32 minutes.

The method

- For each training feature z : run the full $T = 4$ rollout to get $\hat{z}$, push it through the frozen classifier, and minimise $\text{CE}(\mathbf{W}\hat{z} + \mathbf{b}, \mathbf{y})$.
- Backpropagate **through the entire rollout**; update only the flow's parameters.
- Both regularisers the brief suggests are implemented and swept at three strengths each: **displacement** $\|\hat{z} - z\|^2$ and **velocity magnitude** $\|v_k\|^2$.
- Velocities are recovered from the trajectory ($v_k = T(\hat{z}_{k+1} - \hat{z}_k)$) rather than by re-running the network, so the penalty is measured on exactly the states the loss saw.

First: is there anything left to optimise?

The loss is the frozen probe's classification loss **on the probe's own training set** — the same 470 or 1000 k-shot features it was fitted on in Stage 1. Measured at identity, before any training:

Encoder / dataset	train acc	val acc	CE(train)	CE(val)	ratio
ResNet-18 / DTD	98.44%	51.52%	0.327	1.823	7×
DINOv2 / FGVC-Aircraft	100.00%	52.11%	0.005	2.304	449×
ResNet-18 / FGVC-Aircraft	100.00%	27.67%	0.031	3.573	122×

The objective is almost exhausted before training starts. This does *not* mean there is no gradient — the curves show training CE driven down to 10^{-6} . It means **following that signal is memorisation**. The real question Strategy 1 poses is not *can the flow reduce the loss* (it can, completely) but **how little can it be allowed to move before memorisation outweighs whatever real structure it finds**.

The default learning rate was the worst choice tested

config	Δ vs. probe	selected epoch	moved
main (lr 10^{-3})	$+0.15 \pm 0.85$ pp	9	12.7%
lr 10^{-4}	$+1.27 \pm 1.32$ pp	28	28.6%
lr 10^{-5}	$+1.21 \pm 0.91$ pp	131	16.4%
displacement 0.001 / 0.01 / 0.1	$+0.70 / +0.64 / +0.29$	107 / 140 / 105	4.6 / 2.5 / 0.9%
velocity 0.001 / 0.01 / 0.1	$+0.78 / +0.56 / +0.32$	101 / 118 / 68	4.7 / 2.5 / 1.0%

- 10^{-3} is the value Stages 1 **and** 2 both use, and carrying it over was the natural choice — it was right in Stage 2, where a flow trains from scratch against fixed targets.
- Here it is actively harmful: it drives a 657k-788k-parameter network onto 470-1000 features so fast that validation accuracy never rises, and the checkpoint rule salvages an epoch-9 model that has barely moved.
- Reducing the learning rate 10×** is worth about **+1.1 pp** — larger than anything the regularisation achieves. Constraining how *fast* the flow moves beats constraining how *far*.

4 · Step 02 — what the flow is actually exploiting

The step's most interesting result runs against the obvious reading of the diagnostic.

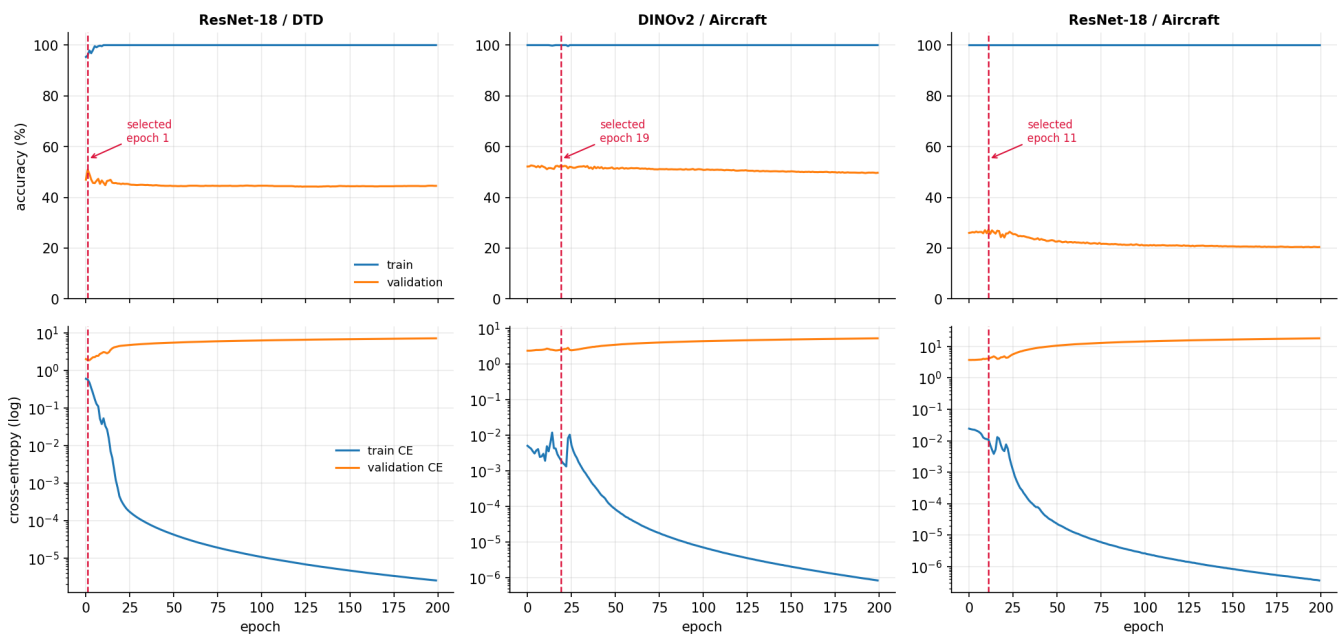
Remaining training loss anti-predicts the gain

Combination	CE(train) at identity	gain at $\text{lr } 10^{-5}$
ResNet-18 / DTD	0.327 — <i>most</i> left to minimise	+0.32 pp — smallest
ResNet-18 / FGVC-Aircraft	0.031	+1.09 pp
DINOv2 / FGVC-Aircraft	0.005 — <i>least</i> left to minimise	+2.21 pp — largest

- The combination with the **most** unminimised loss gains the **least**; the one whose objective is already at zero gains the **most**. Whatever the flow is exploiting, **it is not the residue of the classification objective**.
- Displacement tells the same story from the other side: across all 81 runs, how far the flow moved features correlates **positively** with the gain (Spearman $\rho = +0.54$, $p < 0.001$; +0.70 on DINOv2/Aircraft). The flow has to genuinely move features to help — it just has to get there slowly.
- What we do not claim.** With three combinations it is impossible to say what *does* explain the ordering: DTD differs from the two Aircraft combinations in dataset, class count (47 vs 100) *and* training-set size (470 vs 1000) simultaneously. Stage 2's headroom rule earned its claim because nine cells lined up behind it; three do not support a rule here.

Why validation checkpointing is not a refinement here

Strategy 1: training accuracy is pinned near 100% from the start; validation peaks immediately and decays



Required deliverable. Training accuracy is pinned at $\sim 100\%$ for all 200 epochs — it starts there. Training cross-entropy falls five orders of magnitude, all of it on data already classified correctly. Validation cross-entropy rises monotonically from epoch 0: there is no regime in which both improve.

- Validation accuracy peaks within the first ~ 20 epochs and decays thereafter, so checkpoint selection on validation accuracy is **what makes the method work at all**.
- It is also Stage 1's own rule, which keeps the comparison against the probe consistent.

5 · Step 03 — Strategy 2: classifier-guided targets

99 runs = 11 configurations × 3 combinations × 3 seeds. ~16 minutes.

The method — the brief's six steps

- Roll z through the current flow to get $\hat{z}$.
- Compute the frozen classifier's loss on $\hat{z}$.
- Build an improved $\hat{z}'$ by descending that loss **in feature space**: $\hat{z}' = \hat{z} - \eta \nabla_{\hat{z}} \text{CE}$, unit-normalized so η is the distance actually travelled.
- Source z , target $\hat{z}'$ — **detached**, which is what makes this an ordinary FM update rather than a second end-to-end objective.
- Standard FM: $t \sim U(0,1)$, $z_t = (1-t)z + t\hat{z}'$, $u = \hat{z}' - z$, $\mathcal{L} = \|v(z_t, t) - u\|^2$.
- Recompute targets every `refresh\_every` epochs, as the flow changes.

What the guidance actually produces

The flow can never beat the targets it is aimed at, so those are measured first.

Encoder / dataset	base acc	base CE	target acc ($\eta=1$)	target CE ($\eta=1$)
ResNet-18 / DTD	96.38%	0.594	99.15%	0.316
DINOv2 / FGVC-Aircraft	100.00%	0.005	100.00%	0.001
ResNet-18 / FGVC-Aircraft	100.00%	0.022	100.00%	0.001

On two of three combinations the targets start at 100% accuracy and stay there. The probe already classifies its own k-shot training set perfectly, so there is no *correctness* for the guidance to add. What the gradient still does is collapse the target cross-entropy toward zero — it pushes features into **higher-confidence** regions.

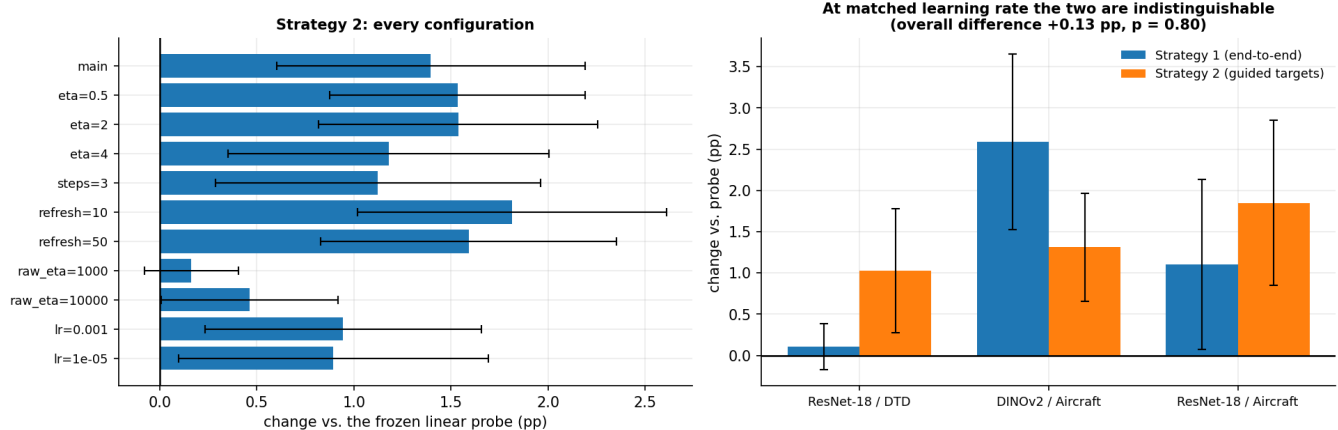
So Strategy 2 is not “move features so the classifier gets them right”. It is a **margin-increasing transform**: move features so the classifier is more certain about answers it already gives. **That it still generalises — +1.3 to +1.9 pp — is the interesting part.**

5 · Step 03 — the four-knob sweep

The brief names exactly four things to experiment with. Each is varied around the main configuration.

config	Δ vs. probe	selected epoch	moved
main ($\eta=1$, 1 step, refresh 1, normalized)	$+1.40 \pm 0.79$ pp	119	17.1%
$\eta = 0.5$	$+1.53 \pm 0.66$ pp	109	12.0%
$\eta = 2$	$+1.54 \pm 0.72$ pp	91	19.5%
$\eta = 4$	$+1.18 \pm 0.83$ pp	103	34.0%
3 target steps	$+1.12 \pm 0.84$ pp	83	29.7%
refresh every 10	$+1.82 \pm 0.80$ pp	116	11.3%
refresh every 50	$+1.59 \pm 0.76$ pp	143	6.1%
unnormalized, $\eta = 10^3$	$+0.16 \pm 0.24$ pp	99	2.1%
unnormalized, $\eta = 10^4$	$+0.46 \pm 0.46$ pp	133	4.3%
$\text{lr } 10^{-3}$	$+0.94 \pm 0.71$ pp	17	10.8%
$\text{lr } 10^{-5}$	$+0.89 \pm 0.80$ pp	144	8.4%

- **Refreshing the targets *less* often is better.** The brief's step 6 says to recompute targets as the flow changes; every epoch gives $+1.40$ pp, every 10 epochs **$+1.82$ pp**. Recomputing constantly means chasing a target that moves with the flow chasing it. Letting it go stale for a few epochs makes the objective closer to stationary.
- **Normalizing the guidance step matters more than any other choice** — $+0.16/+0.46$ pp unnormalized against $+1.4$ to $+1.8$ normalized. The reason is measured: the raw gradient norm differs **100x** across combinations ($7.4e-4$ on DTD, $7.1e-6$ on DINOv2), so one nominal step size means three very different effective steps.
- **Step size is otherwise forgiving** — $\eta \in [0.5, 2]$ all land within 0.15 pp.
- **More target steps is worse** — 3 steps moves features 30% of $\|z\|$ and loses 0.3 pp against 1 step.
- **91 of 99 runs (92%) improve on the frozen probe, and all 11 configurations are positive.**



Every configuration's Δ against the frozen probe, and the strategy comparison at matched learning rate.

6 · Step 04 — The comparison the brief requires

Nothing is retrained here. Every number comes from the run tables steps 02 and 03 wrote.

Classification results

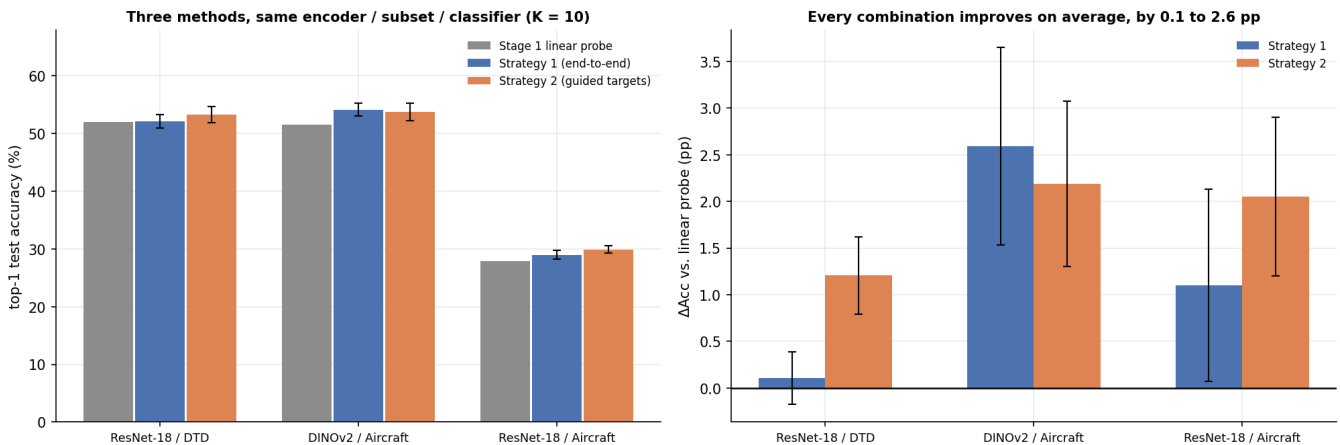
Top-1 test accuracy, K = 10, T = 4, mean $\pm$ sample std over 3 seeds.

Combination	Probe	Strategy 1	Δ Acc	Strategy 2	Δ Acc
ResNet-18 / DTD	52.04%	52.15 $\pm$ 1.15	+0.11	53.24 $\pm$ 1.39	+1.21
DINOv2 / FGVC-Aircraft	51.54%	54.13 $\pm$ 1.13	+2.59	53.73 $\pm$ 1.53	+2.19
ResNet-18 / FGVC-Aircraft *	27.87%	28.97 $\pm$ 0.74	+1.10	29.92 $\pm$ 0.60	+2.05

* beyond the two combinations the brief requires. **Every combination improves on average, under both objectives** — Strategy 1 by +1.27 pp overall, Strategy 2 by +1.82 pp.

How the reported configuration was chosen

- Steps 02 and 03 swept 9 and 11 configurations. Picking the winner by **test** accuracy would make the headline optimistic by construction, so the configuration is chosen by **mean validation accuracy**, which appears in no reported figure.
- Validation and test **independently pick the same configuration** for both strategies (`lr=1e-4` and `refresh=10`), so the numbers above are not a product of test-set selection.



Required deliverable. The three methods per dataset, and Δ Acc against the Stage 1 linear probe.

6 · Step 04 — Do the two strategies actually differ?

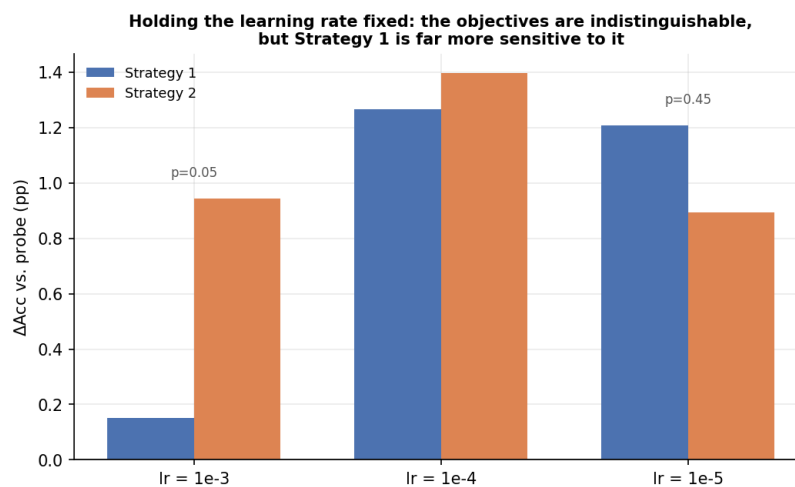
Holding the learning rate fixed isolates the objective from the tuning.

- Strategy 2's main configuration already used $\text{lr } 10^{-4}$, carrying step 02's finding. Strategy 1's main used 10^{-3} . Comparing those two would compare Strategy 2 **with** step 02's lesson against Strategy 1 **without** it — a difference in tuning, not in objective.

lr	Strategy 1	Strategy 2	difference	p
10^{-3}	+0.15	+0.94	+0.79	0.048
10^{-4}	+1.27	+1.40	+0.13	0.802
10^{-5}	+1.21	+0.89	−0.31	0.451

- On accuracy they are indistinguishable** — +0.13 pp at $p = 0.80$ — and they trade wins per combination. Even best-configuration against best-configuration, selected after the fact on *both* sides so the comparison flatters both, the gap is +0.55 pp at $p = 0.30$.
- On robustness they clearly differ, and significantly** ($p = 0.048$ at 10^{-3}). Strategy 1's validation cross-entropy rises far faster on all three combinations, so it has to be stopped much earlier — selected epochs **4 / 66 / 35** against **101 / 117 / 198**.
- The mechanism:** Strategy 1 optimises the classification loss **directly**, so a large step memorises training labels within a handful of epochs. Strategy 2 optimises an **MSE regression onto a target vector** — labels reach it only through where that target was placed, so the same step size cannot memorise them nearly as fast.

Answer to the brief's second question: no. Classifier-guided training does not beat end-to-end rolled-out training on accuracy. It is substantially more robust to the learning rate, and it improves in a higher fraction of runs (**91/99** against **67/81**).

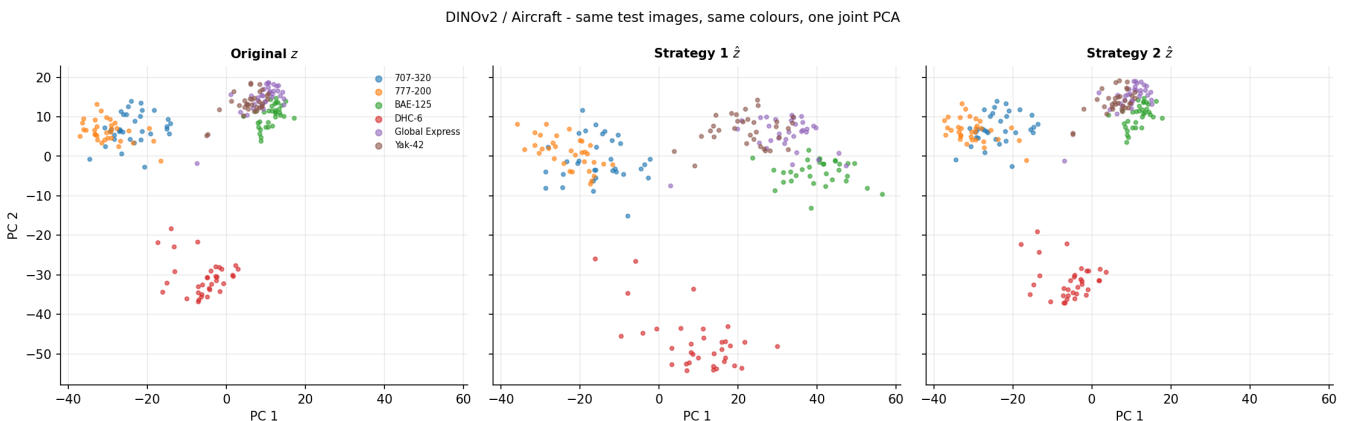


Objective versus tuning: the two strategies at each learning rate.

7 · Step 05 — What the flow does to the feature space

Same test images, same class colours, one PCA fitted jointly over all three sets.

- Three panels per combination — original z , Strategy 1 $\hat{z}$, Strategy 2 $\hat{z}$ — on the **same** test images with the **same** class colours, and **one** PCA fitted on the **concatenation** of all three sets, so the panels are directly comparable rather than three independent rotations.
- **PCA rather than t-SNE**: deterministic, with no perplexity or seed to justify, and “compute the embedding jointly” has an unambiguous meaning for a linear method. Classes are picked by an **even stride** through the sorted class list, not hand-picked.
- Read these as **subset diagnostics**: 6 classes, one seed, 199–240 test images. Two components capture only 20.7–33.8% of the variance — which is exactly why the geometry is *also* measured in the full feature space overleaf.



Required deliverable. DINOv2 / FGVC-Aircraft. Strategy 1 visibly stretches the whole space; Strategy 2's panel is almost indistinguishable from the original — for an equal-or-better result.

The clearest thing the pictures show

DINOv2 / FGVC-Aircraft	displacement $\ \hat{z} - z\ $	Δ accuracy (6 classes)
Strategy 1	34.94	+1.01 pp
Strategy 2	4.60	+1.51 pp

The two strategies reach almost the same accuracy by completely different geometric routes — **equal-or-better result, roughly 8× less movement. How far the flow moves features is not what produces the gain.**

7 · Step 05 — Measuring the geometry properly

A 2-D projection cannot say whether class structure improved. Two quantities can — and they disagree, informatively.

Combination	Strategy	Δ acc (6 cls)	logit margin	Δ separability
ResNet-18 / DTD	1	-1.25	0.997x	+0.0046
ResNet-18 / DTD	2	-0.42	1.92x	+0.0445
DINOv2 / FGVC-Aircraft	1	+1.01	1.58x	-0.0739
DINOv2 / FGVC-Aircraft	2	+1.51	1.64x	+0.0059
ResNet-18 / FGVC-Aircraft	1	+2.01	1.45x	+0.0429
ResNet-18 / FGVC-Aircraft	2	+3.52	1.62x	+0.0103

Logit margin tracks the result

- The frozen classifier is **linear**, so its decision is entirely the gap between the top logit and the runner-up. **Five of six cases widen that gap by 1.45-1.92x.**
- The single exception is **Strategy 1 on DTD at 0.997x** — no widening at all, in fact a hair narrower — and that is precisely the combination where Strategy 1 delivered nothing on the full test set (+0.11 pp, one seed negative). **The diagnostic and the headline agree.**

Class separability does not

- Own-minus-other cosine similarity — the training-free measure from Stage 1 that predicted the whole Stage 1 ordering — moves the **wrong way** where the gain is real: Strategy 1 on DINOv2/Aircraft **loses 0.074** of separability while *gaining* accuracy.
- Stage 2 recorded the same lesson from the other direction: flows there contracted features toward the right prototype without separating them from the wrong ones, and accuracy did not follow.

Neither clustering in a projection nor class-mean geometry is the thing a linear classifier actually consumes — the margin is. This also explains why Strategy 2 needs so little movement: its target vector already points along the direction that widens the margin, so it moves features a short distance in a useful direction. Strategy 1 has no such guidance and must discover a direction by backpropagating through the rollout.

8 · Step 06 — Optional extension: unfreezing the classifier

45 runs = 5 configurations × 3 combinations × 3 seeds. ~13 minutes.

Unfreezing the classifier makes no measurable difference: -0.02 pp, $p = 0.97$. The classifier's extra freedom is available and simply goes unused. Keeping it frozen — which is what makes every other number in Stage 3 interpretable — costs nothing.

The confound, and the control that resolves it

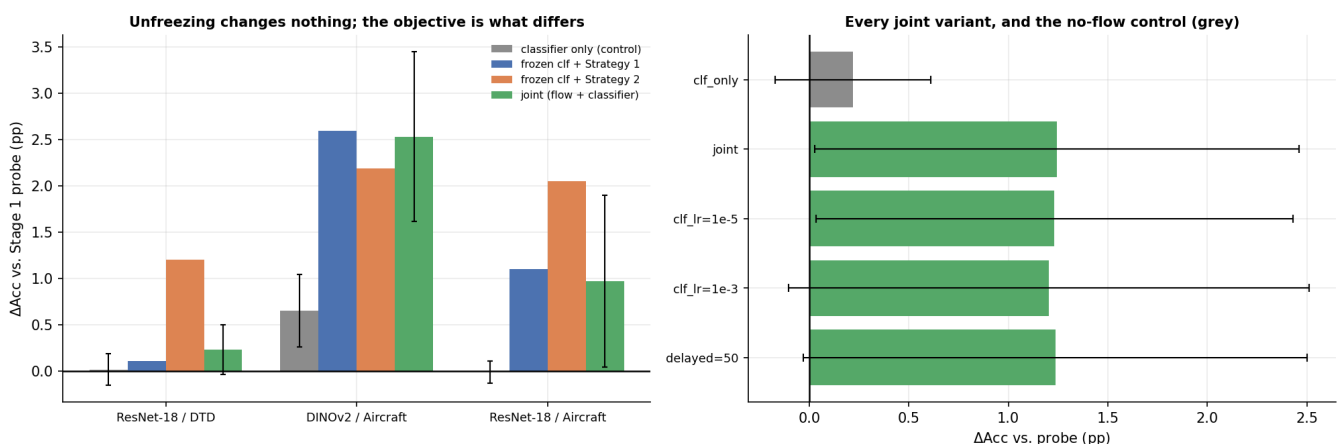
- If *flow + unfrozen classifier* beats the probe there are two explanations: the flow is helping, or **the classifier just got more training**. Stage 1 trained the probe for up to 200 epochs with validation checkpointing so it *should* be at its ceiling — but “should” is not a measurement.
- So this step runs a **classifier-only control**: continue training the classifier from Stage 1's weights, same optimiser, budget and checkpoint rule, **no flow at all**. It is not in the brief; without it the extension cannot be interpreted.

Combination	classifier only	joint	flow contributes	frozen S1	frozen S2
ResNet-18 / DTD	+0.02	+0.23	+0.21	+0.11	+1.21
DINOv2 / FGVC-Aircraft	+0.65	+2.53	+1.88	+2.59	+2.19
ResNet-18 / FGVC-Aircraft	−0.01	+0.97	+0.98	+1.10	+2.05
mean	+0.22	+1.24	+1.02	+1.27	+1.95

Continuing to train the classifier alone moves it **+0.22 pp**, which **independently confirms Stage 1 left the probe at its ceiling** rather than that being assumed. Everything the joint runs gain is therefore attributable to the flow.

The freedom is unused — three ways of seeing it

- The classifier learning rate spans $100\times$ ($10^{-5} \rightarrow 10^{-3}$)** and every result lands between **+1.20 and +1.24 pp**.
- The resulting weight drift spans $50\times$ ($0.004 \rightarrow 0.214$)** with no corresponding change in accuracy.
- `delayed=50`, where the classifier barely moves at all** (drift 0.004), scores identically to letting it move freely from epoch 0.
- Most telling: the classifier drifts **more** when trained alone (0.104) than jointly with the flow (0.039). Once the flow is doing the work, there is nothing left for the classifier to do.



All settings against the probe, and every configuration including the classifier-only control.

9 · What Stage 3 answers

Three questions, their answers, and what limits them.

Does a flow in front of a frozen, trained classifier help?

- **Yes, but modestly: +0.1 to +2.6 pp.** For contrast, Stage 2 found up to **+22.3 pp** applying the same idea in front of an *untrained* prototype classifier.
- **That contrast is the point, and Stage 2 predicted it.** Stage 2's headroom result — FM gain tracks *trained probe* – *untrained prototype*, $\rho = 1.000$ — says the flow recovers roughly what a trained classifier would have gained. In Stage 3 the classifier **is** trained, so that headroom is already spent. What remains is whatever a nonlinear transform can add on top of an already-fitted linear boundary: about a point.

Does classifier-guided training beat end-to-end rolled-out training?

- **Not on accuracy** — +0.13 pp at $p = 0.80$ at matched learning rate.
- **It is substantially more robust**, significantly so ($p = 0.048$), and improves in a higher fraction of runs: 91/99 against 67/81.

What limits both?

- The frozen probe already classifies **its own k-shot training set** at 96–100%, with cross-entropy as low as 0.005. The signal both strategies chase is nearly exhausted before training starts, and following it further is memorisation.
- Both depend entirely on **validation-accuracy checkpointing** — Stage 1's own rule — to stop in time.

Limitations, stated plainly

- **Three seeds give a t-test very little power.** The p -values here are reported for completeness rather than leaned on; the more informative statement is how consistently the sign holds across the full sweeps (83% and 92% of runs).
- **Three combinations cannot support a rule** about *which* combinations benefit. DTD differs from the two Aircraft combinations in dataset, class count and training-set size simultaneously, so the ordering in step 02 is reported without an explanation attached.
- **One K and one T.** The brief fixes both for this stage; the K- and T-dependence measured in Stages 1 and 2 is not re-derived here.
- **The weakest cell is Strategy 1 on DTD** (+0.11 pp, one seed negative), and the step 05 margin diagnostic independently flags the same cell at 0.997×.

Reproducibility

- **52 tests pass** across Stages 2 and 3. ``test_stage3_formulas.py`` re-derives each of the brief's formulas from its wording and asserts the implementation agrees; ``test_probe_equivalence.py`` asserts Stage 3's probe loop matches Stage 1's on accuracy, predictions, selected epoch and every training curve.
- The suite was itself validated by **breaking the implementation eight ways on purpose**. The first run caught only five — the three misses were tests that re-implemented the formula inside the test file and therefore could not fail. After extracting the formulas into the public API, **all eight are caught**.
- Every figure and table here is produced by a tracked notebook, stored with its outputs, and backed by a tracked CSV; each step folder carries an ``_artifact_manifest.csv`` listing exactly what it wrote.
- **Stages 1 and 2 are not modified.** Stage 3 needed the probe's trained weights and a zero-initialised readout; both were added inside ``cvlab3`` rather than by editing published code, and git confirms no Stage 1 or Stage 2 source file differs.